

Volume 03

Core Object System (COS)

Version: 1.0

Status: Draft

Language: English

Chapter 1

Why an Object System?

Traditional security platforms are data-centric.

Project Sentinel shall be object-centric.

Every entity within the platform shall exist as a first-class object with a unique identity, lifecycle, relationships, history and evidence.

Objects are the foundation of the platform.

Everything else builds upon them.

Chapter 2

Definition of an Object

An object represents any real or virtual entity that exists within the cybersecurity ecosystem.

Examples include:

- User
- Organization
- Website
- Server
- Mobile Device
- API
- Domain
- IP Address
- Certificate
- Cloud Account
- Kubernetes Cluster
- Docker Container
- Source Repository
- Employee
- Incident
- Vulnerability
- Threat
- IOC
- Malware Sample

- Email
- Backup
- Router
- Firewall
- Switch
- Database
- Virtual Machine
- Identity
- Secret
- Service Account
- Security Policy

Everything becomes an object.

Chapter 3

Universal Object Structure

Every object shall contain the same logical structure.

Object

This structure is common across all object types.

Chapter 4

Universal Object Identifier

Every object shall receive a globally unique identifier.

Example:

PS-OBJ-0000000001

The identifier never changes.

Even if:

- hostname changes
- IP changes
- owner changes
- location changes

the Object ID remains permanent.

Chapter 5

Object Lifecycle

Every object follows the same lifecycle.

Discovered

Deletion means the object is no longer active; its audit history is retained according to the organization's retention policy.

Chapter 6

Object Types

Initially we define:

Identity Objects

- User
 - Service Account
 - API Key
 - Certificate
 - OAuth Client
-

Infrastructure Objects

- Server
 - VM
 - Container
 - Kubernetes Node
 - NAS
 - Router
 - Firewall
 - Switch
-

Application Objects

- Website
 - API
 - Backend
 - Frontend
 - Mobile App
 - Microservice
-

Cloud Objects

- AWS Account
- Azure Subscription
- Google Cloud Project

- Storage Bucket
 - IAM Role
-

Security Objects

- Vulnerability
 - Finding
 - IOC
 - Malware
 - Alert
 - Incident
-

Business Objects

- Organization
 - Department
 - Employee
 - Supplier
 - Customer
-

Operational Objects

- Scan
 - Mission
 - Report
 - Task
 - Case
-

Chapter 7

Object Relationships

Objects never exist alone.

Example:

Organization

Relationships are typed and directional.

Examples:

- owns
 - hosts
 - connects_to
 - depends_on
 - authenticates_with
 - contains
 - protects
 - monitored_by
 - managed_by
-

Chapter 8

Object History

Objects never lose history.

Each change creates a new event.

Example:

2027

This provides a complete audit trail.

Chapter 9

Object Health

Every object receives a health assessment based on relevant signals.

Examples:

Website

- TLS configuration
- HTTP security headers
- Certificate validity
- Patch level
- Availability

Server

- Patch status
- Running services
- Hardening
- Configuration
- Backup status

Health calculations are transparent and explainable.

Chapter 10

Object Risk

Risk is calculated using evidence collected from multiple sources.

Potential inputs include:

- Vulnerabilities
- Exposure
- Configuration
- Identity posture
- Threat intelligence
- Compliance gaps
- Business criticality

The exact scoring model will be defined in a dedicated Risk Engine volume.

Chapter 11

Object Evidence

Every object stores evidence supporting its current state.

Examples:

- Scan results
- Screenshots
- HTTP responses
- DNS records
- Certificates
- Configuration snapshots
- Event records
- Log excerpts
- File hashes

Evidence should be immutable once stored.

Chapter 12

Object Timeline

Every object has its own timeline.

Example:

Created

Users can replay the history of any object.

Chapter 13

Object Permissions

Access to objects is controlled through Role-Based Access Control (RBAC).

Permissions are assigned to actions (for example: view, modify, approve, delete) rather than directly to users. This makes the authorization model more flexible and easier to audit.

Chapter 14

Object Search

Every object shall be searchable using:

- Name
 - Object ID
 - Tags
 - Type
 - Owner
 - IP Address
 - Domain
 - Status
 - Risk
 - Health
 - Labels
 - Relationships
-

Chapter 15

Object Localization

Objects remain language-independent.

The user interface translates labels, descriptions and messages based on the selected language.

The underlying object model does not change.

New Architectural Decision

PS-ADR-0001

Title

Object-First Architecture

Status

Accepted

Decision

Project Sentinel shall use an Object-First Architecture in which every meaningful entity is represented as a persistent object with identity, relationships, history and evidence.

Rationale

This provides a consistent model across all modules, simplifies integration, and enables advanced capabilities such as Digital Twin, Knowledge Graph, AI-assisted reasoning and comprehensive auditing without creating different data models for each subsystem.

One important adjustment before we continue

I would like to make one architectural refinement.

Earlier we discussed "not using a database." After thinking through the engineering implications, I recommend we rephrase that idea.

A reliable platform **will still need persistent storage**. Instead of saying "no database," I suggest we define:

- **The Object Engine is the primary abstraction.**
- **The persistence layer is an implementation detail.**

That means every module works with objects and never depends directly on a specific storage technology. Whether we later choose PostgreSQL, another database, or a different persistence mechanism is hidden behind the Object Engine. This keeps the architecture clean, portable and realistic while preserving the vision that developers think in terms of objects—not database tables. I believe this is the strongest long-term design because it is both technically sound and fully achievable.

====[USER]=====

Good. Continue

====[ASSISTANT]=====

Excellent. From this point onward, we stop designing isolated modules and start designing **the platform kernel**.

This is exactly how large engineering projects are built. Before scanners, AI, SOC, or Mobile Security, we need a stable platform kernel that everything depends on.

PROJECT SENTINEL BLUEPRINT

Volume 04